

Zapfix Project Architecture and Interview Notes

This PDF summarizes the actual code-backed architecture of the Zapfix project, explains how the diagnosis model works, lists the technologies in use, and includes interview-ready explanations.

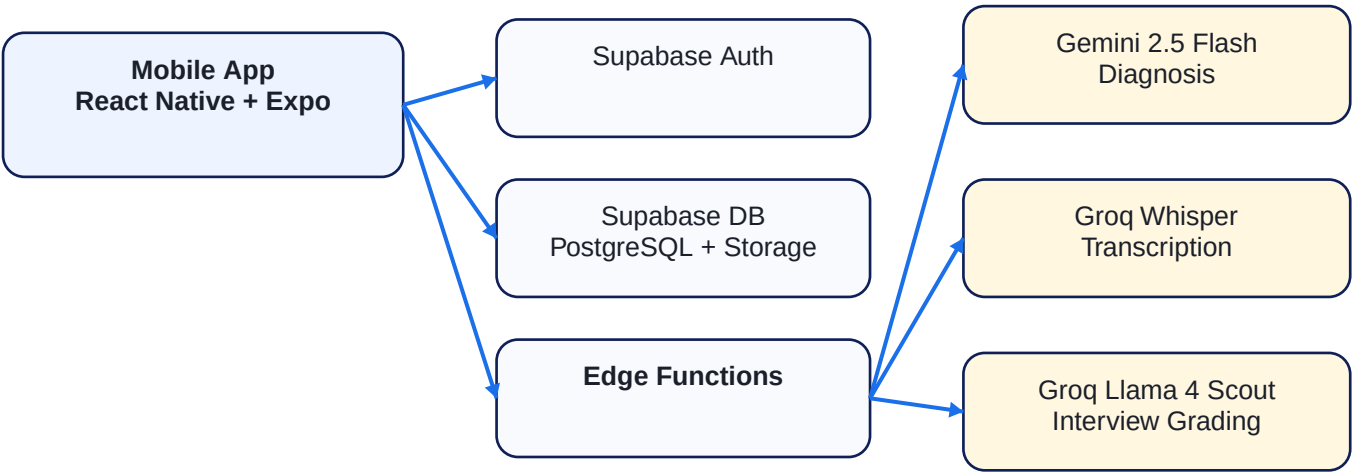
1. Project Overview

Zapfix is an AI-powered home appliance repair platform built as a mobile app. Customers can report an issue using a photo, short video, text description, or voice note. The app diagnoses the issue, estimates the service category, and helps the user move into booking and professional matching.

2. Tech Stack

- Frontend: React Native, Expo, Expo Router, TypeScript, NativeWind, React Query, Zustand
 - Mobile capabilities: expo-image-picker, expo-camera, expo-audio, expo-location, expo-notifications, expo-video-thumbnails
 - Backend: Supabase Auth, Supabase PostgreSQL, Supabase Edge Functions, Supabase Storage
 - Database extensions: PostGIS for geo-spatial data and pgvector for skill embeddings
 - AI services: Gemini 2.5 Flash for diagnosis, Groq Whisper for transcription, Groq Llama 4 Scout for interview grading
 - Payments: Cashfree SDK and Cashfree-backed edge functions

3. High-Level Architecture



Architecture summary:
the app handles UX and capture, Supabase handles auth, storage, and orchestration, and specialized AI models handle diagnosis, transcription, and interview scoring.

4. Diagnosis Model Working

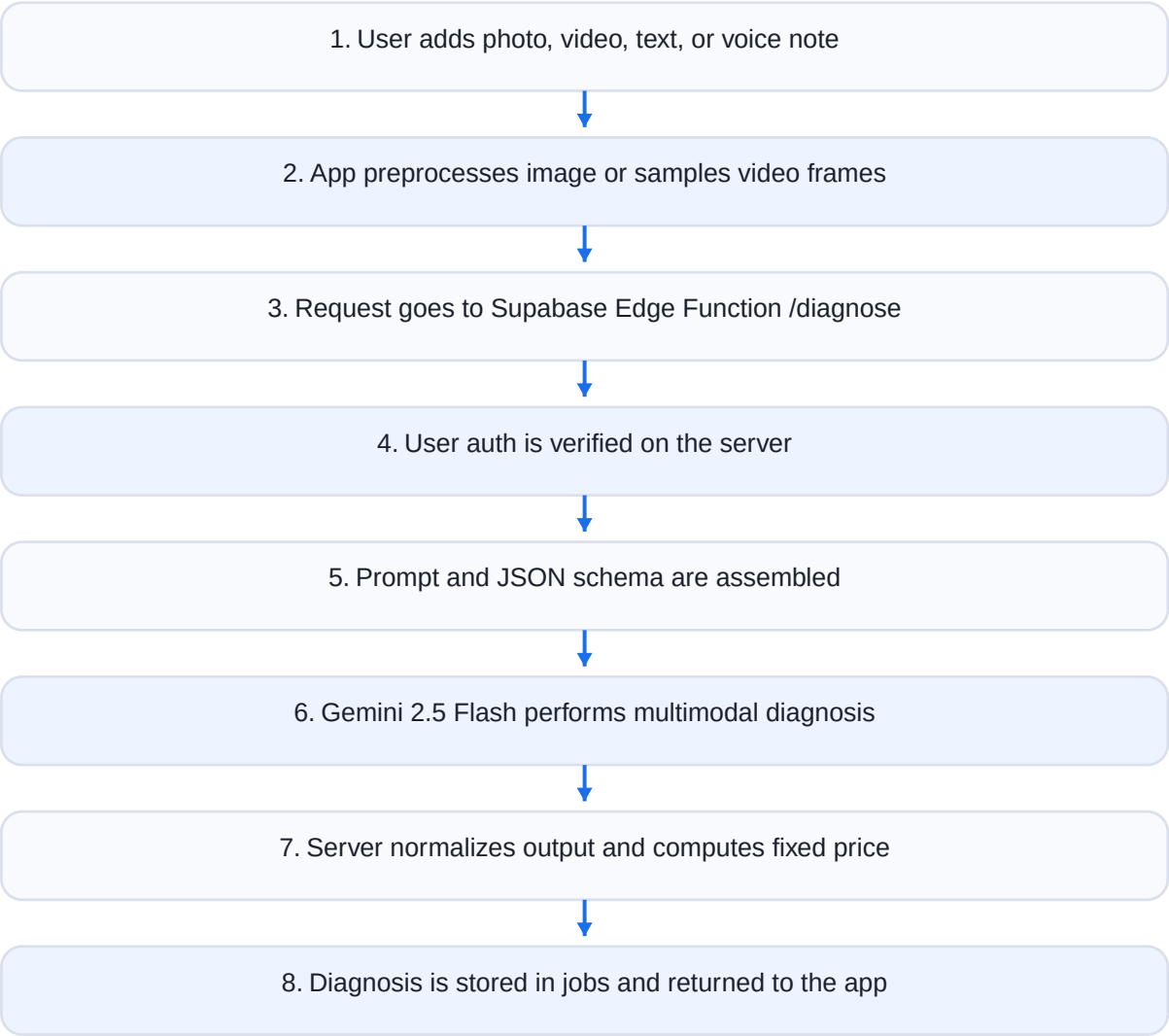
The diagnosis flow is

the core model-driven feature in the project. The customer can provide a photo, short video, typed description, or a voice note. If the input is an image, the app compresses and resizes it before upload. If the input is a video, the app samples several still frames so the model can reason across the clip without being sent a raw video file.

The prepared input is sent to the Supabase Edge Function named `diagnose`. That function authenticates the user, builds a structured multimodal prompt, includes the image frames and customer description, and sends the request to Gemini 2.5 Flash.

The function enforces a strict JSON response schema so the model returns structured fields such as fault name, confidence, required parts, required skill, and urgency. The server then normalizes the output and computes pricing separately using fixed category-based rules.

5. Diagnosis Flowchart



6. Why the Model Design is Good

A strong implementation choice in this project is that the LLM is used for diagnosis, not pricing. The model decides what the likely fault is and which service category is needed, but pricing stays deterministic on the backend. That makes the system much more predictable and easier to trust.



- The model handles fault classification
- The backend handles pricing, validation
- The UI consumes a stable response

7. Voice-to-Text Flow

If the user records a voice note, the

mobile app captures audio using Expo Audio and sends it to the Supabase transcribe edge function. That function forwards the audio to Groq Whisper Large v3 Turbo, receives the transcript, and returns plain text to the app. The transcript is then reused as the problem description for diagnosis.

8. Other AI Flows

- Diagnosis: Gemini 2.5 Flash
- Audio transcription: Groq
- Pro interview grading

9. Database Design

The Supabase schema includes core business entities such as profiles, pro_details, jobs, customer_addresses, pro_skills, pro_inventories,

ry, media assets, notifications, and earnings. The schema also includes geo-spatial points for service locations and vector fields for future skill-based intelligence.

*current_location_geography(Point, 4326)
job_location_geography(Point, 4326)
skill_vector(1536)*

This prepares the system for location-aware dispatching, richer job lifecycle handling, and

future
smart m
atching i
mprove
ments.

10.
End-
to-
End
Flow

- Customer upload
- AI diagnosis
- User conf
- Payme
- Jo
-

